

```
## markdown
# Phase 3 - SQL Analysis
StreamFlix Content Analytics Project
```

```
Name: [Mark Maxwell Louis]
```

The project gave us a file called `schema\_and\_sql.sql` with `CREATE TABLE` statements and 12 queries. I copied those 12 queries into my own file, `Phase3\_SQL\_Queries\_Mark Maxwell Louis.sql` and in this notebook I actually run them against a real database (SQLite) instead of just reading them.

I picked SQLite because it doesn't need a server to be installed - it just runs as a regular file. The SQL itself is the same either way and would also work in PostgreSQL or MySQL without changes.

```
## markdown
## Step 1: Create the database and load the tables
```

```
## code
import sqlite3
import pandas as pd

conn = sqlite3.connect('streamflix.db')

subscribers = pd.read_csv('subscribers_clean.csv')
titles = pd.read_csv('titles_clean.csv')
watch_history = pd.read_csv('watch_history_clean.csv')
ratings = pd.read_csv('ratings_clean.csv')
reviews = pd.read_csv('reviews_clean.csv')
watchlist = pd.read_csv('watchlist_clean.csv')

print('Files loaded.')
```

```
## markdown
SQLite doesn't have a real True/False type like the CREATE TABLE script assumes - it stores booleans as 1 and 0. My CSV files have the words "True"/"False" as text, so I need to convert those columns to 1/0 before loading them into the database.
```

```
## code
subscribers['is_active'] = subscribers['is_active'].map({True: 1, False: 0})
watch_history['completed'] = watch_history['completed'].map({True: 1, False: 0})
titles['is_original'] = titles['is_original'].map({True: 1, False: 0})

print(subscribers['is_active'].unique())
print(watch_history['completed'].unique())
print(titles['is_original'].unique())
```

```
## code
subscribers.to_sql('subscribers', conn, if_exists='replace', index=False)
titles.to_sql('titles', conn, if_exists='replace', index=False)
watch_history.to_sql('watch_history', conn, if_exists='replace', index=False)
ratings.to_sql('ratings', conn, if_exists='replace', index=False)
reviews.to_sql('reviews', conn, if_exists='replace', index=False)
watchlist.to_sql('watchlist', conn, if_exists='replace', index=False)
```

```
# quick check that the tables loaded correctly
check = pd.read_sql('SELECT COUNT(*) AS row_count FROM watch_history', conn)
print(check)
```

```
## markdown
## Step 2: Read the queries from my .sql file
Instead of typing each query out again in Python, I'm reading them straight from `Phase3_SQL_Queries_Mark Maxwell Louis.sql` so there's one single source for the SQL.
```

```
## code
with open('Phase3_SQL_Queries_Mark Maxwell Louis.sql', 'r') as f:
    sql_file_text = f.read()
```

```
# the queries in my file are separated by a blank line + comment starting with "-- Q"
# splitting on ";" gives me each individual query
raw_queries = sql_file_text.split(';')
```

```
# clean up: drop empty strings and add the semicolon back
queries = []
for q in raw_queries:
    q = q.strip()
    if q != '':
```

```

        queries.append(q + ';')

print('Number of queries found in the file:', len(queries))

## markdown
## Step 3: Run each query and look at the result

## markdown
### Q1. Top genres by total watch hours

## code
result_q1 = pd.read_sql(queries[0], conn)
result_q1

## markdown
### Q2. Top 10 countries by watch hours, plus hours per title

## code
result_q2 = pd.read_sql(queries[1], conn)
result_q2

## markdown
### Q3. Engagement by release year

## code
result_q3 = pd.read_sql(queries[2], conn)
result_q3

## markdown
### Q4. Movies vs TV Shows

## code
result_q4 = pd.read_sql(queries[3], conn)
result_q4

## markdown
### Q5. Watch hours by language (top 10)

## code
result_q5 = pd.read_sql(queries[4], conn)
result_q5

## markdown
### Q6. Completion rate by genre

## code
result_q6 = pd.read_sql(queries[5], conn)
result_q6

## markdown
### Q7. Most engaged subscriber segments, by plan

## code
result_q7 = pd.read_sql(queries[6], conn)
result_q7

## markdown
### Q8. Device usage share

## code
result_q8 = pd.read_sql(queries[7], conn)
result_q8

## markdown
### Q9. Churn rate and active rate

## code
result_q9 = pd.read_sql(queries[8], conn)
result_q9

## markdown
### Q10. Monthly Recurring Revenue (MRR) and ARPU

## code
result_q10 = pd.read_sql(queries[9], conn)
result_q10
## markdown

```

```

### Q11. Watchlist conversion rate

## code
result_q11 = pd.read_sql(queries[10], conn)
result_q11

## markdown
### Q12. Content investment efficiency by genre

## code
result_q12 = pd.read_sql(queries[11], conn)
result_q12

## markdown
## Checking my SQL results against my Python results
Q9 gives active rate and churn rate, and Q10 gives MRR and ARPU. These should match
what I calculated with pandas in `Phase3_KPIs_Mark Maxwell Louis.ipynb`.

## code
print('Active rate from SQL:', result_q9['active_rate_pct'][0], '%')
print('Churn rate from SQL:', result_q9['churn_rate_pct'][0], '%')
print('MRR from SQL: $', result_q10['mrr_usd'][0])
print('ARPU from SQL: $', result_q10['arpu_usd'][0])
print()
print('These match the numbers from the Phase 3 KPI notebook, which is a good sign that both are correct')

## code
conn.close()
print('Database connection closed.')

```